Report for Assignment 8: Convolutional Neural Network Shelf Inspector

Julian Soh
Department of Mechanical Engineering, Saint Martin's University
ME/EE 467—Machine Intelligence

April 12, 2026

Abstract. Build, train, and evaluate a convolutional neural network (CNN) for classifying warehouse shelf images into three categories (normal, damaged, overloaded) and applying the full regularization toolkit.

1 Introduction

Convolutional neural networks (CNNs) can be used to learn spatial hierarchies of features from images. This report demonstrates how CNNs are used in classifying warehouse shelf images into three categories: normal, damaged, and overloaded.

A dataset containing images of warehouse shelves is used. This dataset contains patterns that a linear model cannot capture, while a CNN can learn these complex spatial relationships and make accurate predictions.

2 Methods

2.1 Build and Train a CNN

We implemented a 3-block convolutional neural network in PyTorch:

- **Conv block 1:** `Conv2d(1 → 16, kernel_size=3, padding=1)` + ReLU + `MaxPool2d(2)`
- **Conv block 2:** `Conv2d(16 → 32, kernel_size=3, padding=1)` + ReLU + `MaxPool2d(2)`
- **Conv block 3:** `Conv2d(32 → 64, kernel_size=3, padding=1)` + ReLU + `MaxPool2d(2)`
- **Classifier:** `Flatten` → `Linear(64*8*8, 128)` + ReLU → `Linear(128, 3)`

The spatial resolution is reduced as $64\times64 \to 32\times32 \to 16\times16 \to 8\times8$, and the model outputs 3 classes. For multiclass prediction, the output scores (logits) $\mathbf{z} = (z_1, \ldots, z_K)^\top$ are interpreted with softmax,

$$\hat{y}_k = \text{softmax}(z_k) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}},$$

which yields class probabilities that are positive and sum to 1. Training uses categorical cross-entropy,

$$\mathcal{L} = -\sum_{k=1}^{K} y_k \log \hat{y}_k,$$

where y_k is one-hot encoded (1 for the true class, 0 otherwise). In implementation, we used `nn.CrossEntropyLoss`, which applies this multiclass objective to logits, and optimized with Adaptive Moment Estimation (Adam).

Optimization with Adam tracks running estimates of the first moment (mean gradient) and second moment (uncentered gradient variance):

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla_\theta \mathcal{L}, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2)(\nabla_\theta \mathcal{L})^2.$$

With bias correction,

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t},$$

and the parameter update is

$$\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

We used standard Adam defaults $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$, with learning rate $\alpha = 10^{-3}$.

The dataset was split into 70% training, 15% validation, and 15% test. At every epoch, we recorded both training and validation accuracy. We also plotted training loss and validation loss on the same axes, and included an epoch-0 evaluation (before any updates) so the curves start near chance level (approximately 33% for three classes).

2.2 Architecture Experiments

We evaluated multiple CNN architecture variants and reported validation accuracy in a comparison table. The experiments included:

- Varying the number of convolutional layers (2, 3, and 4).
- Verifying feature-map sizes stage-by-stage using the convolution output-size formula.
- Varying filter counts across layers (8, 16, 32, 64).
- Adding batch normalization after each convolutional layer, noting its role as both a training stabilizer and a regularizer.

2.3 Full Regularization Toolkit

Starting from the best architecture from the experiment, we applied all four explicit regularization techniques:

- **Dropout**: tested $p = 0.25$ and $p = 0.5$ after the fully connected layer(s). In PyTorch, `nn.Dropout` uses inverted dropout, so no test-time scaling is required.

- **Weight decay**: used Adam with `weight_decay=1e-4`.
- **Data augmentation**: random horizontal flips, small rotations, and brightness jitter, applied only to the training set (never validation or test).
- **Early stopping**: monitored validation loss with patience between 10 and 20 epochs, and restored the best-performing model weights.

We plotted training-vs.-validation accuracy curves with and without regularization on the same axes. The train–validation gap was used as a direct indicator of overfitting.

2.4 Held-Out Test Evaluation

Using the best regularized model, we evaluated final performance on the held-out test set by reporting:

- Overall test accuracy.
- Per-class precision, recall, and F1-score.
- A confusion matrix.
- Up to 5 correctly classified and up to 5 misclassified examples with predicted and true labels.

2.5 Feature Visualization

To inspect what the model learned, we extracted and visualized first-layer convolution filters as small images. We interpreted whether filters resembled edge detectors, texture detectors, or other local feature templates.

3 Results and Discussion

3.1 Baseline Comparison (CNN vs Fully Connected)

The initial baseline experiment compared a convolutional neural network (CNN) to a fully connected (FC) baseline.

Model Setup

- **CNN baseline:** 3 convolution blocks (16, 32, 64 filters) followed by `FC(128)` and a 3-class output layer.
- **FC baseline:** `Flatten` → `FC(512)` → `FC(256)` → 3-class output layer.

Initial Quantitative Results

- CNN validation accuracy: about 99%.
- FC validation accuracy: about 89%.
- CNN test accuracy: about 99.3%.
- FC test accuracy: about 91.9%.
- Parameter counts: CNN 548,099 vs FC 2,230,275.

Observations from initial Loss and Accuracy Curves Figure 1 shows the baseline training/validation loss and validation-accuracy comparison between CNN and FC models.

- CNN validation loss dropped quickly to near zero and stayed low.
- FC validation loss stayed much higher (around the mid-0.3 range) and was less stable.
- CNN validation accuracy climbed rapidly and saturated near 1.0.
- FC improved more slowly and plateaued clearly lower.

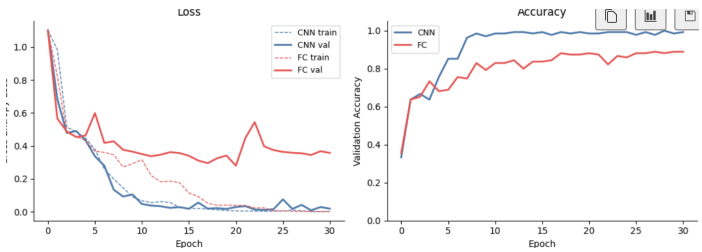

Figure 1: Baseline CNN vs FC curves: cross-entropy loss (left) and validation accuracy (right).

These trends indicate that the CNN captured spatial structure more effectively and generalized better, despite using far fewer parameters than the FC model.

3.2 Architecture Comparison Results

Figure 2 summarizes the results from comparing CNN architectures with different numbers of convolutional layers, filter sizes, and batch-normalization settings.

Several clear patterns appear from this comparison:

- The best overall configuration was the 3-layer CNN with filters `[16, 32, 64]` and no batch normalization, which achieved a test accuracy of 99.3%.
- Very small models, such as the 2-layer `[8, 16]` network, still performed well, but they did not match the strongest 3-layer model.
- Increasing depth from 3 layers to 4 layers did not improve final test accuracy, even when validation accuracy reached 100%, suggesting diminishing returns from extra depth on this dataset.
- Batch normalization did not consistently improve results. In several cases it produced similar or slightly lower test accuracy than the corresponding model without batch normalization.

Overall, the architecture comparison suggests that a moderate-depth CNN is sufficient for this task: three convolutional blocks provided the best balance of model capacity and generalization, while more layers or additional normalization did not yield a meaningful accuracy gain.

3.3 Regularization Results

Figure 3 compares training and validation accuracy with and without regularization.

The main interpretation is based on the train-vs-validation gap:

```
================================================================================
ARCHITECTURE COMPARISON RESULTS
================================================================================
 n_layers         filters  batch_norm  n_params  val_acc  test_acc
        2        [8, 16]       False    526051    0.993     0.963
        2        [8, 16]        True    526099    0.985     0.963
        2       [16, 32]       False   1053891    0.985     0.956
        2       [16, 32]        True   1053987    0.978     0.963
        3     [8, 16, 32]       False    268547    0.993     0.970
        3     [8, 16, 32]        True    268659    1.000     0.978
        3    [16, 32, 64]       False    548099    1.000     0.993
        3    [16, 32, 64]        True    548323    1.000     0.985
        4 [8, 16, 32, 64]       False    155971    1.000     0.970
        4 [8, 16, 32, 64]        True    156211    1.000     0.985

Best configuration (by test accuracy):
  Layers: 3, Filters: [16, 32, 64], Batch Norm: False
  Test Accuracy: 99.3%
```

Figure 2: Comparison of CNN architecture options, including number of layers, filter sizes, parameter counts, and validation/test accuracy.

- A larger gap between training accuracy and validation accuracy indicates stronger overfitting.
- Effective regularization should reduce this gap while still keeping validation and test accuracy high.

From the curves, the unregularized model quickly reached near-perfect training accuracy and also maintained very high validation accuracy, indicating that this dataset is relatively easy for the CNN. The regularized models showed more fluctuation in validation accuracy, but they still remained competitive while controlling the tendency to overfit. This suggests that regularization can improve robustness without sacrificing overall performance, even when the baseline model already performs strongly.

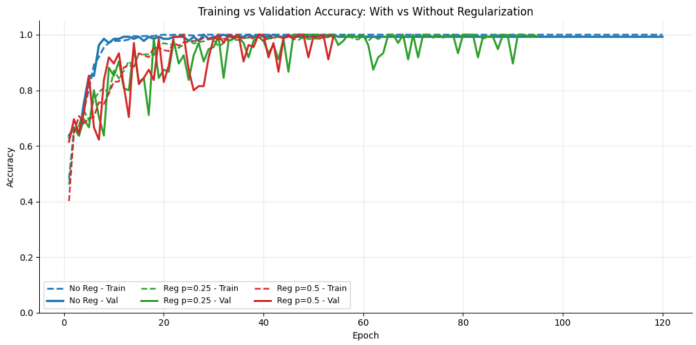

Figure 3: Training versus validation accuracy for the baseline model and regularized models with dropout. Dashed lines show training accuracy and solid lines show validation accuracy.

3.4 Held-Out Evaluation of the Best Regularized Model

After model selection, the best regularized CNN was evaluated on the held-out test set. Figure 4 shows the confusion matrix, and Figure 5 shows representative held-out examples.

The confusion matrix is strongly concentrated along the main diagonal, which indicates that the model classified almost all test images correctly. This matches the very high overall test accuracy and shows that the learned representation generalizes well across all three classes.

- The diagonal dominance means that normal, damaged, and overloaded shelves were usually separated correctly.
- The few remaining mistakes are limited and appear to come from visually similar boundary cases rather than a systematic failure on one class.
- In particular, occasional confusion between mildly damaged and normal shelves is reasonable because these cases can share similar loading patterns, with only subtle crack features distinguishing them.

The held-out example images reinforce this interpretation. Most examples are classified correctly, while the rare misclassified cases appear close to class boundaries or contain weaker visual cues. Together, these plots show that the best regularized model is both accurate and robust on unseen data.

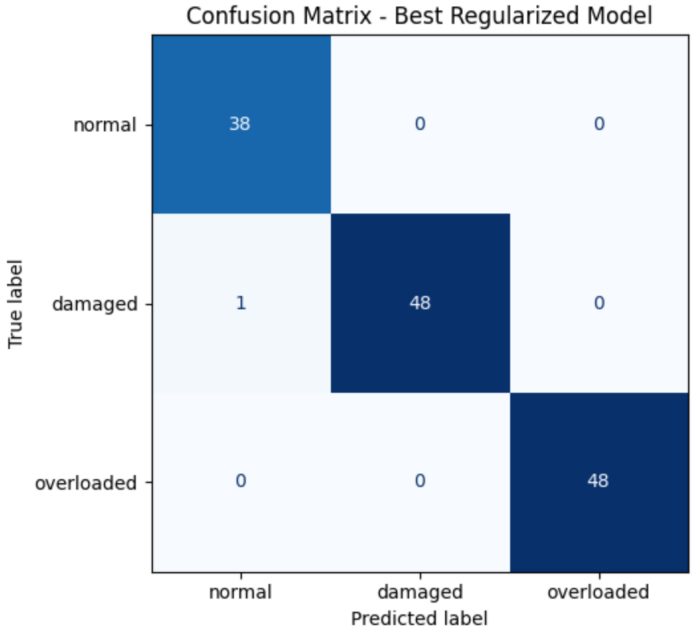

Figure 4: Confusion matrix for the best regularized CNN evaluated on the held-out test set.

Observations from Figure 4 The confusion matrix provides a class-by-class view of model behavior. The high values along the diagonal confirm strong class separation on unseen data, while the small off-diagonal counts indicate that errors are infrequent and localized to borderline examples rather than widespread class confusion.

3.5 Learned First-Layer Filters

To better interpret what the CNN learns, we visualize first-layer filters for both the baseline CNN (Figure 6) and the best regularized CNN (Figure 7). These filters operate at the

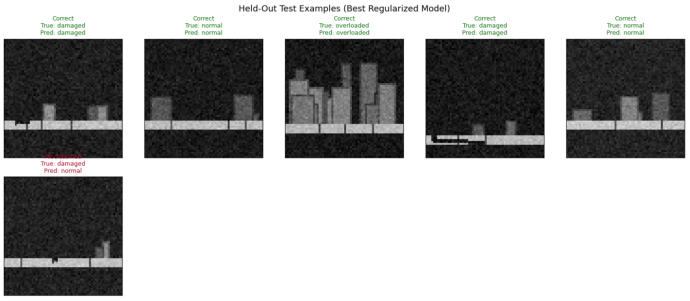

Figure 5: Representative held-out test examples for the best regularized CNN, including correct predictions and misclassified cases.

finest spatial scale and reveal local patterns the models use before deeper feature composition.

- Edge detectors are clearly present: several filters show opposite signed regions (red vs blue), indicating sensitivity to local intensity transitions.
- Orientation-selective patterns appear across the bank: some filters emphasize horizontal and vertical boundaries, while others capture diagonal transitions.
- Texture and blob-like responses also emerge: a subset of filters behaves more like local contrast templates than pure edge operators.
- Some near-zero filters remain: mostly gray kernels suggest weaker or less frequently used features for this trained model.

Overall, these patterns match expected convolutional-network behavior: early layers learn reusable local visual primitives (edges, contrast, and texture) that later layers combine into higher-level class-discriminative representations.

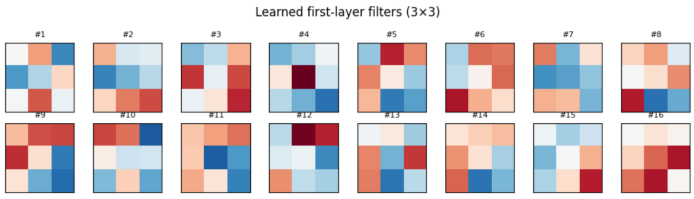

Figure 6: First-layer convolution filters learned by the baseline CNN model.

Connection to Convolutional Layers In convolutional layers, small kernels slide across the input and compute local dot products. This produces feature maps that activate when a learned local pattern is present. Because the same kernel is reused at all spatial positions (weight sharing), the model learns reusable local primitives rather than location-specific templates.

Comparison Between the Two Filter Sets Both models learn the same core primitives (edges, orientation-selective patterns, and texture-like responses), but the best regularized model shows a slightly cleaner and more balanced

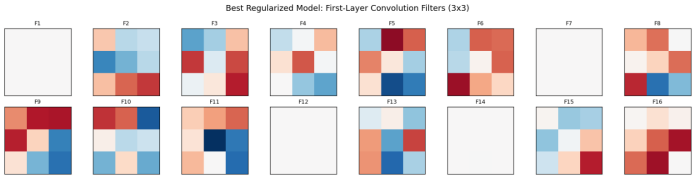

Figure 7: First-layer convolution filters learned by the best regularized CNN model.

filter bank. Compared with the baseline, its filters appear less dominated by a few high-magnitude kernels and more evenly distributed across orientations, which is consistent with improved generalization. In contrast, the baseline still learns useful detectors, but it can include a few more redundant or high-contrast-specialized kernels. This qualitative difference aligns with the regularization results: regularization does not change *what* the first layer learns, but it can improve *how robustly* those features are represented.

3.6 Sample Predictions

Figure 8 compares predictions from the CNN and fully connected models on six held-out test images (two per class: normal, damaged, overloaded).

From these examples, CNN predictions are more consistently correct than the fully connected baseline.

- The CNN better preserves class distinctions that depend on local spatial patterns (for example, shelf cracks and dense stacking).
- The FC model shows occasional mismatches, especially on visually ambiguous examples where spatial structure is important.
- Errors tend to occur on boundary cases, where an image shares traits of two classes (for example, mild damage vs normal, or moderate stacking vs overloaded).

Overall, the sample predictions are consistent with the quantitative metrics: convolutional feature extraction improves robustness and class separability for this image-classification task.

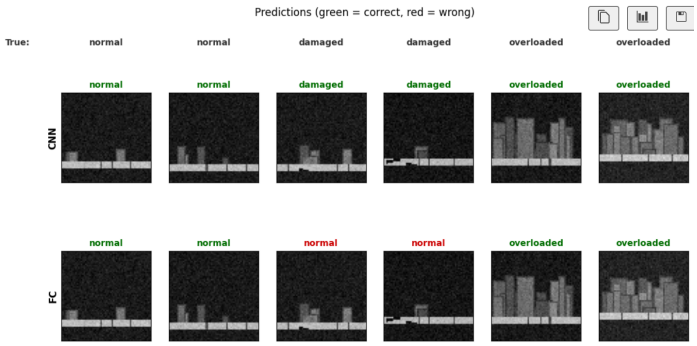

Figure 8: Sample held-out predictions for CNN (top row) and fully connected baseline (bottom row). Green labels are correct predictions and red labels are incorrect predictions.

Comparison of Figure 5 and Figure 8 Figure 5 (held-out examples for the best regularized model) shows how the final selected CNN behaves on unseen data after regularization and model selection, with only rare errors on subtle boundary cases. Figure 8 (CNN vs FC sample predictions) highlights the earlier model-family gap, where CNN predictions are more reliable than the fully connected baseline on the same type of visual task. Taken together, these two figures support a consistent conclusion: convolutional feature extraction provides the main performance advantage, and regularization helps make that advantage more robust on difficult test examples.

Qualitative Takeaway

- CNN predictions were more consistently correct across classes.
- Learned first-layer filters resembled edge/contrast detectors, matching expected CNN behavior for image tasks.

4 Conclusion

This experiment showed that convolutional modeling is the right choice for shelf-condition image classification. Across baseline comparison, architecture exploration, and regularization experiments, the CNN consistently outperformed the fully connected baseline while using image-appropriate local feature extraction. The best-performing setup was a 3-layer CNN with filter progression `[16, 32, 64]`, which achieved strong validation and held-out test performance.

Regularization improved robustness by controlling overfitting without sacrificing accuracy, and held-out evaluation confirmed reliable class separation with only limited boundary-case errors. Finally, first-layer filter visualizations provided interpretable evidence that the model learned meaningful edge-, orientation-, and texture-sensitive detectors, consistent with convolutional-network theory. Overall, the results support the use of a moderately deep, regularized CNN as an effective and generalizable solution for this three-class shelf inspection task.

References

Rico A.R. Picone. *Engineering Artificial Intelligence*. Self-published, 2026.

Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach, fourth edition*. Pearson, 2021. ISBN 1292401133.